# BQ BigQuery Cheat Sheet & Notes

Date: 10 / 10 / 2026
Page No: 01

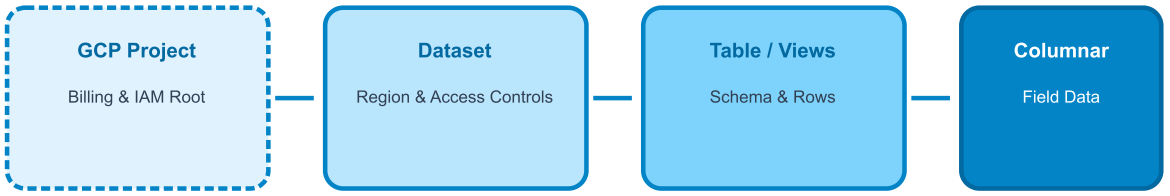
## 1 What is BigQuery?

- Google Cloud's fully-managed, serverless data warehouse.
- Designed for ultra-fast SQL queries on Petabyte-scale data.
- Separates compute (Dremel) from storage (Colossus) for independent scaling.
- Includes built-in ML, Geospatial GIS, and Python analysis.

## 2 Key Architecture Components

- **Colossus:** Google's distributed columnar storage engine.
- **Dremel Engine:** Turns SQL into execution trees across thousands of slots.
- **Jupiter Network:** 100 Gbps network connecting compute and storage.
- **Borg System:** Allocates hardware compute resources dynamically.

## 3 Quick Visual Guide: BigQuery Resource Hierarchy




## 4 Partitioning

Divides tables based on Date, Timestamp, or Integer range. Prunes unused data to cut query cost.

## 5 Clustering

Sorts data inside partitions by key columns (up to 4). Speeds up FILTER / GROUP BY operations.

## 6 Slots & Billing

Slots = Virtual CPUs. Choice of **On-Demand** ($ per TB scanned) or **Capacity** (Slot commits).

📌 **Golden Rule for BigQuery:** Never use `SELECT *`. Always select explicit columns to avoid scanning unwanted data blocks!

## 7 Cost Optimization Best Practices

- Use **Dry Run** to check scanned bytes before running queries.
- Preview tables via UI for FREE data checks.
- Set maximum bytes billed limits to prevent runaway scripts.
- Utilize **Partitioning & Clustering** on large tables.
- Cache query results for repeated identical runs.

## 8 Data Governance & PII Security

- **Column-Level Security:** Restrict access using Policy Tags.
- **Data Masking:** Hash or mask sensitive attributes (e.g. `675****`).
- **Authorized Views:** Share aggregated query outputs without granting access to underlying tables.
- **Encryption:** Default CMEK/Google-managed keys.

## 9 Partitioning vs. Clustering Comparison

| Feature | Partitioning | Clustering |
|---|---|---|
| **Best Used For** | Cost estimation, filtering by dates/ranges. | High-cardinality columns, exact matching & grouping. |
| **Cost Savings** | Directly reduces scanned bytes before execution. | Improves performance; indirectly reduces scanned bytes. |
| **Limit** | 1 column per table (Date/Timestamp/Integer). | Up to 4 ordered columns per table. |

## 10 Top BigQuery Interview Questions & Answers

### Q1: How does BigQuery differ from traditional RDBMS?

BQ is serverless, columnar, and scales compute/storage independently. RDBMS relies on row-based storage and fixed instance sizing.

### Q2: When should you combine Partitioning and Clustering?

When tables are >1 GB, partitioned by date/timestamp, and frequently filtered or aggregated on secondary key columns (e.g., user_id).

### Q3: How do you handle duplicate data in BigQuery?

Use `ROW_NUMBER() OVER(PARTITION BY id ORDER BY timestamp DESC)` and filter for `rn = 1`, or merge using staging tables.

### Q4: What are Slots in BigQuery?

Slots are virtual CPUs used to execute SQL queries. BigQuery dynamically allocates slots based on query complexity and workload demand.

## 11 Advanced Interview Questions & Practical Scenarios

### Q5: Explain the difference between Wildcard Tables and External Tables.

**Wildcard Tables** query multiple BigQuery native tables sharing a prefix using `FROM `table_*``. **External Tables** query data directly stored outside BigQuery (e.g., CSV/Parquet files in GCS) without loading it into BQ storage.

### Q6: What is a "Resources Exceeded" query error, and how do you fix it?

Occurs when a query demands more memory than allocated per slot (often caused by expensive JOINs or huge `ORDER BY` clauses). Fix by partitioning/clustering, avoiding SELECT *, or replacing `COUNT(DISTINCT)` with `APPROX_COUNT_DISTINCT()`.

### Q7: How does caching work in BigQuery, and when is it bypassed?

BigQuery stores query results in a temporary cached table for 24 hours at no cost. Caching is bypassed if nondeterministic functions are used (e.g., `CURRENT_TIMESTAMP()` or `RAND()`), or if target table data changes.

### Q8: How do you implement Row-Level and Column-Level Security in BigQuery?

**Row-Level Security** uses Row Access Policies with `CREATE ROW ACCESS POLICY` filtering by user groups. **Column-Level Security** uses Taxonomy Policy Tags assigned to columns via IAM roles.

🎓 **Interview Pro-Tip:** Always mention **Separation of Compute & Storage** and **Cost Optimization** (Partitioning/Clustering) when answering any BigQuery system design question!